State-Grounded Masked Language Generation for Persistent Editable Worlds

DKI-7
xiaohua@dki-7.com

Abstract

Language generation in persistent worlds differs from ordinary text continuation. Autoregressive language models are strong at left-to-right sequential generation, but in a long-running, editable, multi-user world system, language is distributed across many surfaces, including dialogue, quest logs, character memories, rumors, chronicles, and world records. When a local world-state change occurs, the system must locate the affected text regions, preserve unrelated content, synchronize multiple related language surfaces, and support conflict merging and rollback when necessary. Relying only on sequential continuation or long prompting can easily cause over-rewriting, missed updates, setting drift, and state inconsistency.

This paper proposes a State-Grounded Masked Language Generation framework for persistent editable worlds. The framework takes structured world state as input, maps world-state changes to affected semantic-language fields, masks these local regions, generates candidate replacements, validates their consistency, and writes the accepted result back into an updated world state. We formalize the system through three layers: World State, Latent Language Field, and Surface Text. We define the task as Persistent World State-Grounded Text Regeneration. The task emphasizes that generation should not rewrite text from scratch, but should perform local repair, linked updates, state validation, and bidirectional synchronization within an existing world state.

To make the problem observable and evaluable, we design a small benchmark scenario named VillageWorld-Edit. The scenario uses a frontier village with NPCs, locations, social relations, events, quest logs, dialogues, rumors, and chronicles. It includes four editing tasks: local relation modification, event insertion, multi-user conflict merging, and rollback. The experimental design compares autoregressive language models, retrieval-augmented autoregressive generation, plain masked editing, and the state-grounded masked method. The evaluation metrics include consistency, edit locality, preservation rate, propagation correctness, conflict resolution quality, rollback fidelity, hallucination rate, and state update accuracy.

The central claim of this paper is that masked or diffusion-style language generation should not be understood only as a faster decoding technique. Instead, it can serve as a generation paradigm for local, multi-point, rollback-aware text synchronization in persistent worlds. The contributions of this paper are: defining persistent world state-grounded text regeneration; proposing a state-aware mask strategy based on world-state deltas and dependency graphs; introducing bidirectional synchronization between world state and language fields; and designing a small dynamic social-world benchmark for validating the paradigm. This framework provides an initial methodological basis for language generation, world-memory maintenance, and multi-user editing consistency in LSJ / Li Shijie-like persistent world systems.

Keywords: persistent world; masked language generation; state-grounded generation; text regeneration; world state; language field; LSJ; multi-user editing

1 Introduction

1.1 Research Background: From Text Continuation to Persistent-World Language Synchronization

Large language models have greatly improved the efficiency of text generation, dialogue, narrative drafting, and quest-description generation. Traditional autoregressive language models generate text by predicting the next token from left to right, which makes them naturally suitable for story continuation, question answering, summarization, and one-shot content generation. However, in a persistent world system, language generation does not operate on an isolated text fragment. It operates on a long-running world that is continuously edited and that contains both structured state and multiple language surfaces.

Taking LSJ / Li Shijie as an example, language in the world does not exist only in a single story passage or a single dialogue. It may be distributed across NPC memories, player-facing dialogue, quest logs, village rumors, world chronicles, system records, and event summaries. When a player modifies an NPC relationship, inserts an event, revokes an action, or when multiple users submit mutually conflicting world settings, the system must update not only the next sentence, but also a set of language regions related to world state.

Language generation in persistent worlds is therefore closer to a state synchronization problem. Given a world-state change, the system should find the affected semantic-language fields, locally rewrite the necessary content, keep unaffected content stable, and realign the generated result with structured world state. This paper summarizes the problem as follows: autoregressive language models are optimized for sequential text continuation, while persistent worlds require localized, editable, rollback-aware, and state-consistent language regeneration.

1.2 Limitations of Autoregressive Language Models in Persistent Worlds

The core advantage of autoregressive language models is continuous generation. They can produce fluent text from previous context and can incorporate world settings, character relations, and recent events into a prompt to produce locally plausible outputs. However, when a world system requires local, multi-point, traceable state updates, this approach exposes structural limitations.

First, autoregressive generation tends to rewrite too much content. For a local relation update, such as changing "the hunter trusts the village chief" to "the hunter suspects the village chief," the ideal behavior is to update only the hunter's memory, chief-related dialogue, inn rumors, and a small amount of quest text. In practice, an autoregressive model may reorganize whole passages and alter unrelated settings.

Second, autoregressive generation has difficulty propagating linked updates reliably. Text surfaces in a persistent world depend on each other. Inserting an event may affect several NPC memories, quest logs, rumors, and chronicles. Even if a prompt asks the model to "update everything," the system can still miss some locations or generate inconsistent statements across surfaces.

Third, autoregressive generation does not naturally support arbitrary-position editing. Many world texts are local fields inside an existing text collection, not a trailing context waiting for continuation. Left-to-right continuation is not a natural mechanism for synchronous repair across multiple positions.

Fourth, autoregressive generation does not directly support rollback and conflict merging. A persistent world must record which fields were affected by an event and restore or regenerate them when the event is revoked. Multi-user editing may also introduce mutually exclusive claims, requiring explicit conflict localization and compatible regeneration.

These limitations do not imply that autoregressive models cannot participate in persistent-world language generation. They show that a paradigm centered only on sequential continuation is insufficient to describe this problem.

1.3 Research Problem: State-Change-Driven Local Text Regeneration

This paper studies the following question: in a persistent editable world, how can a system locally, consistently, and rollback-safely regenerate multiple related language surfaces according to world-state changes?

Unlike ordinary text generation, the input here is not a single natural-language context. It includes world state with entities, locations, relations, events, timelines, memories, rules, and conflict records. The output is also not just a text fragment. It includes updated surface text, updated latent semantic-language representation, and state changes that can be written back into the world system.

We call this task Persistent World State-Grounded Text Regeneration. The key requirements are locality, consistency, propagation, preservation, rollbackability, and mergeability. Locality means modifying only affected language fields. Consistency means that generated text must agree with world state. Propagation means all related text surfaces are updated. Preservation means unrelated settings remain stable. Rollbackability means the system can track the influence range of a state change and support undo. Mergeability means multi-user conflicts can be explicitly localized and expressed in compatible language.

1.4 Contributions

This paper proposes a State-Grounded Masked Language Generation framework for persistent editable worlds. The main contributions are as follows.

First, we define Persistent World State-Grounded Text Regeneration, reframing persistent-world language generation from one-shot text continuation into local text regeneration driven by structured world-state changes.

Second, we formalize the system through three layers: World State, Latent Language Field, and Surface Text, and clarify the bidirectional relationship between text generation and state writeback.

Third, we propose a state-aware mask strategy. Instead of randomly masking tokens, the strategy uses world-state deltas and dependency graphs to locate affected semantic-language fields and regenerate them locally.

Fourth, we design VillageWorld-Edit, a small dynamic social-world scenario for evaluating local relation modification, event insertion, multi-user conflict merging, and rollback in persistent-world editing.

Fifth, we introduce evaluation metrics for persistent-world language synchronization, including consistency, edit locality, propagation correctness, rollback fidelity, state update accuracy, and hallucination rate.

2 Related Work and Methodological Background

2.1 Autoregressive Language Models and Sequential Generation

Autoregressive language models formulate text generation as a conditional probability decomposition: `p(x_t | x_<t)`

Under this paradigm, the model predicts subsequent tokens from already generated or provided previous context. This method is strong in open-ended text generation, dialogue, summarization, and code generation, and it forms the basis of many current large language model applications. For one-shot narrative generation or quest-description drafting, an autoregressive model can absorb character settings, world background, and style requirements through prompts and output coherent text.

However, the default structure of autoregressive generation is sequential continuation. It is suitable for answering "what should be written next," but not naturally suited for answering "which existing positions in the world should be locally modified." When the text object expands from a single document to multiple language surfaces in a persistent world, generation is no longer just context continuation. It must maintain

consistency among world state, text fields, and historical records.

2.2 Masked / Diffusion-style Language Generation

Masked language generation and diffusion-style language generation provide a view of generation different from sequential continuation. They usually start from masked or noisy text representations and recover, denoise, or reconstruct the target text through iterative processes. Compared with left-to-right generation, these methods naturally support infilling, editing, and non-sequential position generation.

This capability is important for persistent-world scenarios. World-state changes often affect only part of the text, while unaffected regions should remain stable. Masked generation allows the system to explicitly mark positions that need rewriting, concentrating generation on affected fields and avoiding regeneration of unrelated content. Multiple related positions can also be masked at the same time, allowing the system to update dialogue, memory, quest logs, and rumors together.

This paper does not claim that masked or diffusion-style language models are universally superior to autoregressive models. It focuses on a narrower and clearer question: in persistent, editable, multi-surface world generation, whether mask-based generation is structurally better suited for local repair, multi-point synchronization, and rollback.

2.3 Continuous Embedding Flow and ELF

Recent discussions around Embedded Language Flows (ELF) suggest that language generation can be performed in continuous embedding space through continuous-time flow matching and then mapped back to discrete tokens in the final stage. Such work shows that language generation does not have to be fully constrained by left-to-right paths over discrete tokens. Continuous, flow-based, and diffusion-style modeling has become an important direction.

From the perspective of this paper, the significance of ELF-like work is not that it directly solves the persistent-world problem. Rather, it provides methodological background: language can be treated as an editable, transformable, locally repairable representation space. Language surfaces in a persistent world have long-term dependencies with structured state. If state changes, local masks, and consistency constraints can be represented in a latent language field, the update mechanism can become more stable than pure prompt-based continuation.

2.4 Text Editing, Infilling, and Local Regeneration

Text editing and infilling have shown that models can fill missing fragments or replace local content under a given context. However, ordinary text-editing tasks usually operate on a single document or text segment and focus on fluency and semantic plausibility after editing. Local regeneration in persistent worlds has stronger system constraints.

First, edit positions should not be manually chosen arbitrarily; they should be inferred automatically from world-state changes. Second, edited objects may be distributed across multiple files, fields, and character memories. Third, edited results must be consistent with structured state and may in turn affect that state. Fourth, the editing process must record its influence range to support future rollback and conflict audit.

For this reason, this paper extends ordinary masked editing into state-grounded masked generation. The difference is that ordinary masked editing mainly targets text-internal consistency, while state-grounded masked generation targets consistency among world state, language fields, and long-term memory.

2.5 Persistent Worlds, World State, and Multi-surface Text Synchronization

State in a persistent world includes not only space, objects, and rules, but also character relations, event history, organizational structures, quest progress, and memory records. Language text is the external interface of this state. An NPC dialogue may reflect memory, a quest log may reflect event progress, a village rumor may reflect a social relation change, and a chronicle may record events that have occurred.

This means that text is not an external description of the world. It is part of world state. Text and state have a bidirectional relationship: state changes require text updates, and generated text may produce new structured state or state corrections. Persistent-world language generation must therefore handle both language-surface synchronization and world-state writeback.

2.6 Summary: The Gap Between Existing Methods and This Problem

Existing autoregressive language models are strong at generating fluent text. Masked and diffusion-style methods are strong at local recovery and non-sequential editing. Continuous embedding flow suggests that language generation can operate in more flexible representation spaces. However, these directions do not directly answer the key question in persistent worlds: when world state changes, how can a system automatically locate affected language fields, rewrite only necessary positions, synchronize multiple text surfaces, validate consistency, and write the result back into world state?

This paper addresses this gap through State-Grounded Masked Language Generation. It expands the target of masking from random tokens or manually selected fragments to semantic-language fields driven by world-state changes and dependency graphs.

3 Task Definition and Formalization

3.1 Persistent World State-Grounded Text Regeneration

This paper defines language updating in persistent worlds as Persistent World State-Grounded Text Regeneration. Given an existing world state and a state change, the system must locate affected language fields, locally regenerate these fields, and output updated text together with an updated world state.

This task differs from ordinary language generation in three important ways. First, the input includes structured world state rather than only natural-language context. Second, the output includes multiple text surfaces and writable state rather than only a single text segment. Third, the goal is not simply to maximize continuation fluency, but to balance local editing, state consistency, and long-term maintainability.

3.2 World State, Latent Language Field, and Surface Text

We divide the system into three layers:

```
S: World State
Z: Latent Language Field
X: Surface Text
```

World State (S) represents structured world state, including entities, locations, relations, events, timelines, memories, rules, and conflicts. It provides the basis for judging world facts, character relations, and event history.

Latent Language Field (Z) represents an intermediate semantic-language field. It is not the final text shown directly to players. Instead, it connects structured state and surface text. Z may include editable semantic fields, text slots, relation descriptions, event summaries, and generation candidates.

Surface Text (X) represents user-visible or player-visible text, including dialogue, quest text, event logs, chronicles, NPC memory text, and rumor text. X is the external language expression of world state.

Traditional autoregressive language models mainly model:

```
p(x_t | x_<t)
```

Ordinary masked language generation mainly models:

```
p(x_clean | x_masked / x_noisy)
```

This paper focuses on:

```
p(Z, X | S, C)
S' = Update(S, X, Z)
```

Here, C denotes the editing context, including user operations, event sources, conflict information, permission boundaries, and current task goals. The formula indicates that the model does not merely output text; it also participates in latent language-field updating and world-state writeback.

3.3 Inputs, Outputs, and State Updates

The input of the task includes the initial world state S, the state change ΔS, the existing latent language field Z, the existing surface text X, and the editing context C. The state change may come from player actions, system events, external real-world inputs, multi-user editing, or rollback requests.

The output includes the updated latent language field Z', the updated surface text X', the updated world state S', and a set of auditable editing records. The editing records should at least contain the masked fields, generation candidates, accepted version, consistency-checking result, and state-writeback result.

This input-output design allows the system to answer three questions: which text fields are affected by the state change; how these fields are updated; and whether the updated text changes the world state.

3.4 State Changes, Dependencies, and Affected Text Fields

A state change does not directly equal a text change. The system needs a dependency graph to determine which language fields are affected. The dependency graph can connect entities, relations, events, locations, quests, and text fields. For example:

```
relationship_changed(A, B)
→ mask dialogue(A)
→ mask memory(A)
→ mask memory(B)
→ mask rumor_entries
→ mask chronicle_related_sentence
```

In a frontier-village scenario, if "the hunter trusts the village chief" is changed to "the hunter suspects the village chief," then the hunter's personal memory, chief-related dialogue, rumors known by the innkeeper, and quest descriptions related to the chief may be affected. However, pharmacy inventory, forest weather descriptions, and unrelated NPC backgrounds should not be modified.

Affected-field detection is therefore a key step of the method. It determines the generation scope and whether the system can keep unrelated content stable.

3.5 Differences from Traditional AR Continuation and Plain Masked Editing

Compared with traditional autoregressive continuation, this task does not ask the model to continue from the end of a text. It asks the model to perform local, multi-point, constrained regeneration inside an existing world. Generation positions are determined by state changes and dependency graphs, not by text order.

Compared with plain masked editing, the mask in this paper is not selected randomly or manually. It is triggered by a world-state delta. The generated result must be consistent not only with textual context, but

also with structured world state, and it must support state writeback.

State-Grounded Masked Language Generation therefore places language generation inside a world-state maintenance loop. Text is no longer only an output; it becomes part of the runtime state of the world.

4 State-Grounded Masked Language Generation Framework

4.1 Framework Overview

The proposed State-Grounded Masked Language Generation framework contains five main stages:

```
World State S
→ Detect world state delta
→ Mask affected semantic-language fields
→ Regenerate local text / latent fields
→ Validate consistency
→ Write back updated state S'
```

The core idea is that the generation system should not rewrite world text from scratch. Instead, it should locate the regions that need modification inside an existing world state, regenerate these regions locally, and reintegrate the language result into the world system through consistency validation and state writeback.

4.2 World State Delta Parsing

World State Delta represents a change in world state. It can originate from player operations, system events, multi-user editing, external inputs, or rollback requests. Typical deltas include relation changes, event insertion, location-state changes, quest-progress changes, character-memory changes, and conflict declarations.

The system first converts natural-language or structured operations into processable state changes. For example, the user input "change the hunter's attitude toward the village chief to suspicion" can be parsed as:

```
relation_update(
  subject = Hunter,
  object = Village_Chief,
  relation_type = trust,
  old_value = trust,
  new_value = suspicion
)
```

Only after the state change is structurally represented can later masking and validation have a stable basis.

4.3 Affected Semantic-Language Field Detection

Affected-field detection maps a state change to the language fields that need updating. The system can maintain a dependency graph that connects world entities, relations, events, and text fields. Edges in the graph indicate that a text field depends on a certain state fact.

For example, a relation change between the hunter and the village chief may affect:

```
Hunter.memory.about_chief
Hunter.dialogue.greeting
Chief.dialogue.about_hunter
Innkeeper.rumor.current
```

`QuestLog.village_conflict`
`Chronicle.recent_events`

This detection mechanism allows the system to explicitly control the editing scope. Fields detected as affected enter the mask set, while unaffected fields remain frozen context.

4.4 State-Aware Mask Policy

The state-aware mask policy is the key difference between this framework and ordinary masked editing. Ordinary masked editing often determines mask regions by text position or user selection. In contrast, this paper selects mask regions according to world-state changes, dependency graphs, and task goals.

The mask policy should satisfy four requirements. First, coverage: all key fields affected by the state change should be included. Second, locality: unrelated fields should not be masked. Third, hierarchy: the system may mask both latent language fields and surface text. Fourth, auditability: each mask decision should be traceable to a specific state change or dependency relation.

For multi-user conflicts, the mask policy should also mark conflict regions. For example, if user A claims "the traveling merchant is the thief" while user B claims "the traveling merchant was framed," the system should mark text fields related to the merchant, the pharmacy theft, evidence sources, and village rumors as conflict regions awaiting resolution.

4.5 Local Regeneration and Candidate Generation

The local regeneration stage generates candidates only for masked fields. The generation process can use a masked language model, a diffusion-style language model, an LLM prompt simulator, or a hybrid pipeline. The key issue is not the exact model type, but the fact that generation is constrained to affected fields and explicitly grounded in world state.

Candidate generation can output multiple versions. Each candidate should include text content, associated state fields, generation rationale, and possible new state facts. For example, for a "pharmacy theft" event, the system may generate different rumor versions, but each version must specify its relation to event state, NPC knowledge scope, and evidence strength.

4.6 Consistency Validation and Conflict Handling

Consistency validation determines whether candidate text agrees with world state. Validation rules may include entity consistency, relation consistency, timeline consistency, quest-state consistency, memory-visibility consistency, and anti-hallucination constraints.

For example, if world state has not confirmed that the traveling merchant is the thief, the system should not write in the chronicle that "the traveling merchant stole the medicine." A more appropriate expression is "rumors in the village implicate the traveling merchant, but the evidence remains incomplete." This preserves the conflict while avoiding turning an unconfirmed state into a fact.

Conflict handling should not simply average multiple user inputs. It should structure conflicts explicitly. The system can write conflicting claims into a conflicts field and generate surface text that is compatible with the current evidence state.

4.7 Text-to-State Writeback

Generated text may introduce new state information. A quest log may confirm that an NPC has learned about an event; a dialogue line may introduce a new suspect; a chronicle entry may fix the time of an event. The system needs to extract structured facts from Z' and X' and write them back to S'.

State writeback must be constrained. Not every rhetorical expression should become a world fact. The system needs to distinguish facts, rumors, hypotheses, subjective character beliefs, and unconfirmed clues. Only validated state updates should enter the core world state. Other content can enter auxiliary fields such as rumor, belief, or conflict.

4.8 Rollback and Multi-user Editing Support

A persistent world must support undo and audit. Because this framework records the world-state delta, masked fields, generation candidates, and state-writeback results, it can locate the language fields affected by a previous event during rollback and regenerate or restore them.

In multi-user editing, different users may modify related states simultaneously. This framework supports merging through conflict-field detection and local masks. The system does not need to rewrite the entire world; it regenerates only the language and state regions related to the conflict.

State-Grounded Masked Language Generation is therefore not only a text generation process. It is a language-state maintenance mechanism for persistent worlds.

5 VillageWorld-Edit Benchmark Scenario and Construction

5.1 Why a Small Dynamic Social Scenario Is Needed

If masked generation is discussed only abstractly as support for infilling and editing, the contribution remains too general. To verify whether the method is genuinely suitable for persistent worlds, we need a scenario that is small and controllable, yet still contains real linked dependencies.

This paper uses a frontier village as the prototype of VillageWorld-Edit. A village is limited in scale and easy to inspect manually, while still containing NPCs, locations, social relations, events, quests, rumors, and chronicles. It is therefore complex enough to expose the difficulties of persistent-world language synchronization.

5.2 World State Schema of the Frontier Village

The world state of VillageWorld-Edit can include the following fields:

```
entities: NPCs, organizations, objects
locations: village gate, inn, pharmacy, forest, shrine
relations: trust, suspicion, cooperation, hostility, debt
events: occurred events, unconfirmed events, revoked events
timeline: event order and time markers
memories: NPC personal memories and visibility scopes
rumors: unconfirmed rumors and their sources
quests: quest state, objectives, and triggers
chronicle: world chronicle and public records
conflicts: multi-user editing conflicts
```

The goal of this schema is not to simulate a complete world. It provides a minimal evaluable object for state-grounded text regeneration.

5.3 NPCs, Locations, Relations, Events, and Text Surfaces

The initial scenario can include five NPCs: the village chief, the hunter, the pharmacist, the innkeeper, and the traveling merchant. Locations include the village gate, inn, pharmacy, forest, and shrine. Social

relations include the chief trusting the pharmacist, the hunter suspecting the traveling merchant, and the innkeeper knowing village rumors.

The text surfaces include:

```
dialogue_texts.json: NPC dialogue snippets
memory_entries.json: NPC memory entries
quest_log.md: quest log
chronicle.md: village chronicle
rumor_board.md: village rumors
world_state.json: structured world state
```

These text surfaces together form the language objects that the generation system must maintain. A single state change may affect several files or fields at the same time.

5.4 Four Editing Tasks

Task 1 is local relation modification. The input changes "the hunter trusts the village chief" into "the hunter suspects the village chief." The system needs to update the hunter's memory, chief-related dialogue, inn rumors, quest descriptions, and world records, while preserving pharmacy information, forest descriptions, and unrelated NPC backgrounds.

Task 2 is event insertion. The input is "the pharmacy was robbed last night." The system needs to infer who knows about the event, who is suspected, which NPC dialogues change, how the quest log is extended, and how village rumors are updated.

Task 3 is multi-user conflict merging. User A writes "the traveling merchant is the thief," while user B writes "the traveling merchant was framed." The system must mask the conflict region and generate a compatible expression, such as "rumors in the village accuse the traveling merchant of stealing medicine, but the pharmacist finds that the evidence is incomplete."

Task 4 is rollback. If the player revokes the "pharmacy theft" event, the system should restore the pharmacy state, NPC dialogues, quest descriptions, rumors, and chronicle. The mask method can explicitly identify positions affected by that event and regenerate or restore them.

5.5 Data Files and Example Organization

The initial version of VillageWorld-Edit can use lightweight JSON and Markdown files:

```
world_state.json
dialogue_texts.json
memory_entries.json
quest_log.md
chronicle.md
rumor_board.md
edit_cases.json
dependency_graph.json
gold_outputs.json
```

The file edit_cases.json records state-change inputs. The file dependency_graph.json records dependencies between state fields and text fields. The file gold_outputs.json records ideal outputs revised or annotated by humans.

5.6 Consistency Checking Rules

Consistency checking rules are used to evaluate generation results automatically or semi-automatically. They may include:

```
whether an entity exists in world_state
whether relation descriptions match current state
whether the event timeline is contradictory
whether an NPC knows information outside its visibility
whether rumors are incorrectly written as facts
whether revoked events remain in public records
whether unrelated text is unexpectedly modified
whether generated text introduces unauthorized new settings
```

These rules make VillageWorld-Edit not only a demonstration, but also a benchmark for comparing generation methods.

6 Experimental Design and Evaluation Metrics

6.1 Experimental Goals

The experiment does not aim to prove that masked or diffusion-style language models outperform autoregressive models on all text generation tasks. It aims to verify whether, in persistent, editable, multi-surface world generation tasks, a state-grounded masked method provides better locality, consistency, propagation, and rollback ability.

Experiments should be built around the four task types in VillageWorld-Edit: local relation modification, event insertion, multi-user conflict merging, and rollback. Each task should compare different methods under the same initial world state and the same editing input.

6.2 Baseline Settings

This paper recommends four baselines.

The first is an AR LLM baseline. This method directly puts the world state and modification request into a prompt and asks an autoregressive model to generate the updated text. It represents the most direct application of a large language model.

The second is an AR + RAG baseline. This method first retrieves world memories and text fragments related to the state change, then asks an autoregressive model to generate the result. It tests whether retrieval augmentation is sufficient for multi-surface synchronization.

The third is a Plain MDLM / masked editing baseline. This method masks or locally edits text, but does not use structured world-state deltas or dependency graphs. It tests whether the advantage comes merely from masking.

The fourth is the State-grounded masked method. This method uses world-state deltas and dependency graphs to determine mask regions, then performs local generation, consistency validation, and state write-back.

6.3 State-Grounded Masked Method Setup

The experimental pipeline of the state-grounded masked method is:

```
input world_state_delta
→ find affected semantic fields
→ mask affected text fields
```

```
→ generate replacement candidates
→ consistency checker
→ state updater
→ output updated world state + updated texts
```

At the initial stage, it is not necessary to train a new model. A large language model can simulate different generation modules. The key is to fix the pipeline, mask strategy, dependency graph, and evaluation protocol to validate the task definition and framework.

6.4 Automatic Evaluation Metrics

Consistency Accuracy measures whether generated text is consistent with world state. For example, it checks whether the text turns an unconfirmed rumor into a fact or incorrectly describes NPC relations.

Edit Locality measures whether the method modifies only the fields that should be modified. An ideal method should avoid rewriting unrelated text.

Preservation Rate measures the preservation of unaffected settings. This metric captures setting drift often seen in autoregressive generation.

Propagation Correctness measures whether all related text is updated. For example, it checks whether a relation change is synchronized to dialogue, memory, quests, and rumors.

Conflict Resolution Score measures whether multi-user conflicts are merged reasonably. A high-quality result should preserve conflict sources and evidence state instead of arbitrarily choosing one side.

Rollback Fidelity measures whether the system restores the correct state and text after an event is revoked.

Hallucination Rate measures whether generated results introduce unsupported settings, characters, objects, or events.

State Update Accuracy measures whether generated text can be correctly written back into structured world state.

6.5 Human Evaluation Protocol

Automatic metrics cannot fully cover world coherence and narrative naturalness, so human evaluation is needed. Human evaluators can score outputs along three dimensions: world consistency, textual naturalness, and editing acceptability.

World consistency focuses on whether text matches known world state. Textual naturalness focuses on fluency and whether character voice is reasonable. Editing acceptability focuses on whether the modification satisfies the user intent, whether it over-edits, and whether it misses key propagation.

To reduce subjective bias, human evaluation should anonymize method outputs, shuffle the results of different baselines, and provide the same initial world state and editing input.

6.6 Error Taxonomy

In addition to quantitative metrics, the paper should include an error taxonomy. Typical errors include:

```
over-editing: modifying unrelated fields
under-propagation: missing fields that should be updated
state contradiction: conflict between text and structured state
rumor-fact collapse: writing rumors as facts
memory leakage: an NPC knows information it should not know
rollback residue: revoked events remain after rollback
unsupported hallucination: adding unsupported settings
```

`conflict flattening: flattening multi-user conflicts too aggressively`

The error taxonomy helps explain the value of state-grounded masking and reveals where the current framework still needs improvement.

7 Discussion

7.1 Why This Paper Is Not Merely About a Decoding Trick

State-Grounded Masked Language Generation should not be understood as a faster decoding trick. Its core goal is not to reduce the number of generation steps, but to change how language is maintained in persistent worlds. In a persistent world, language text and structured state coexist over the long term. World changes require local text updates, and generated text may in turn modify world state.

This paper therefore focuses on whether a generation paradigm is suitable for world-state maintenance. Autoregressive models can still serve as candidate generators, but left-to-right continuation alone cannot express affected-field detection, local masking, consistency validation, and state writeback.

7.2 Structural Advantages of Masked Generation in Persistent Worlds

The advantage of masked generation comes from its structural match with persistent-world editing needs. Persistent worlds often require arbitrary-position editing, multi-field synchronization, and preservation of unaffected content. A mask mechanism allows the system to explicitly mark positions that need updating and constrain generation to those positions.

This advantage is especially clear in four tasks: local relation modification requires avoiding overrewriting; event insertion requires multi-point synchronization; multi-user conflict merging requires conflict-region marking; and rollback requires finding the fields affected by a past event and restoring or rewriting them. For these tasks, masked generation more naturally expresses where to repair, where not to repair, and why repair is needed.

7.3 Boundaries with Pure AR, RAG, and Ordinary Text Editing

Pure autoregressive methods are useful for quickly generating fluent single passages, but in multi-surface state synchronization they can miss updates or over-edit. RAG can provide relevant context, but retrieving related content does not determine which fields must be modified, nor does it ensure that generated results can be written back into state.

Ordinary text editing can improve local modification ability, but without world state and dependency graphs, it still does not know which text fields are affected by a given state change. The boundary of the proposed method is that mask decisions must come from world-state changes, not from text positions alone.

This paper does not reject AR, RAG, or ordinary masked editing. They can serve as candidate generation or retrieval modules inside the framework. The key claim is that these modules need to be placed inside a state-grounded pipeline to satisfy the system requirements of persistent worlds.

7.4 Current Limitations

This paper is currently closer to a system and framework paper than to a completed new neural-model paper. It proposes task definitions, formal objects, mask strategies, and an experimental benchmark, but early experiments can first use large language models to simulate generation modules without immediately training a new MDLM or diffusion language model.

Another limitation is the cost of constructing dependency graphs and state schemas. As a persistent world becomes more complex, dependencies among entities, events, relations, and text fields become harder

to maintain. Future work should study automatic dependency discovery, learnable affected-field detectors, and stronger consistency checkers.

Text-to-state writeback also has semantic ambiguity. Subjective character beliefs, rumors, facts, and conflict declarations must be strictly distinguished; otherwise, the system may incorrectly write uncertain expressions into core world state.

7.5 Significance for LSJ / Li Shijie

For LSJ / Li Shijie, this framework provides a way to incorporate language generation into the persistent-world runtime loop. The goal of LSJ is not to generate one-shot content, but to maintain a long-running, editable, collaborative world. Language text in the world must continuously update with state changes and remain consistent with NPC memories, quests, social relations, and world history.

State-Grounded Masked Language Generation can serve as a synchronization layer between LSJ world memory and language surfaces. It enables local modification, event insertion, conflict merging, and rollback without destroying the existing world, providing a methodological basis for language consistency in persistent worlds.

8 Conclusion and Future Work

8.1 Summary

This paper proposes State-Grounded Masked Language Generation to address language regeneration in persistent editable worlds. Unlike ordinary autoregressive continuation, the paper focuses on local, multi-point, rollback-aware text synchronization triggered by world-state changes.

The paper formalizes the system through World State, Latent Language Field, and Surface Text; defines Persistent World State-Grounded Text Regeneration; and designs a state-aware mask strategy based on world-state deltas and dependency graphs. Through affected-field detection, local regeneration, consistency validation, and state writeback, the framework places text generation inside a persistent-world state maintenance loop.

8.2 Main Contributions

The contributions of this paper are fourfold. First, it defines a state-grounded text regeneration task for persistent worlds. Second, it proposes a formal relationship among world state, latent language fields, and surface text. Third, it introduces a generation framework that selects mask regions according to state changes. Fourth, it designs the VillageWorld-Edit small dynamic social scenario and corresponding evaluation metrics for comparing AR, RAG, plain masked editing, and state-grounded masked methods.

Together, these contributions show that language generation in persistent worlds should not be treated only as text output. It should be treated as part of world-state synchronization and maintenance.

8.3 Future Work: From Framework Validation to Model Training

Future work can proceed in three directions. First, VillageWorld-Edit should be expanded with more complete world_state.json, dialogue_texts.json, memory_entries.json, quest_log.md, and chronicle.md files, together with human-annotated gold outputs.

Second, masked or diffusion-style language models should be trained or fine-tuned for this task so that they can directly receive structured state, mask fields, and consistency constraints rather than relying only on prompt simulation.

Third, automatic dependency-graph construction, state-consistency checking, and text-to-state write-back should be studied. As world scale grows, manually maintaining dependencies will become difficult, and the system will need more automated semantic detection and auditing mechanisms.

8.4 Extensions Toward LSJ Persistent Worlds

In LSJ / Li Shijie, this framework can be extended to more complex social structures, long-term NPC memory, multi-user collaborative world building, real-world event ingestion, and cross-modal content synchronization. Future systems will need not only to update text, but also to link text changes with quests, assets, spaces, NPC behavior, and world rules.

The final claim of this paper is that persistent worlds need not only one-shot generation capability, but a long-term maintainable generation mechanism. The value of mask / diffusion-style generation is not merely to replace autoregressive models, but to provide a more suitable paradigm for world-state-driven local repair, multi-point synchronization, and rollback-aware editing.